

The background features several overlapping circles of varying sizes and shades of gray. In the top right corner, there is a target symbol consisting of three concentric circles.

Application au plus haut niveau

Introduction à l'IA
par SAMIR MILOUDI

Introduction à l'IA par la pratique

Objectifs pédagogiques

Introduction

Présentation

Algorithme d'apprentissage

Implémentation d'une régression linéaire supervisée

Contexte d'utilisation

Exemple Java – Prédiction des retards de livraison

Configuration du projet Java

Développement

TP Pour aller plus loin

Solution

Lexique

Annexe - Outils IA dans l'écosystème Java

- ❖ **Comprendre les bases de l'intelligence artificielle**
 - Comprendre les concepts clés de l'IA :
 - Machine Learning / Deep Learning
- ❖ **Identifier des cas d'usage pertinents (ex : logistiques)**
- ❖ **Distinguer IA classique et IA générative**
- ❖ **Maîtriser l'implémentation d'un modèle IA en Java**
 - Mettre en œuvre un pipeline ML complet en Java
 - Mettre en œuvre un pipeline DL complet en Java
- ❖ **Mettre en œuvre un TP de prédiction logistique en Java**
 - Manipuler des outils IA compatibles Java (Smile, DL4J, Weka, Tribuo, etc.)

Prérequis

Avant de commencer, les étudiants doivent :

- Maîtriser les bases de Java
- Maîtriser les bases de Maven
- Avoir un environnement de développement préparé (IDE comme IntelliJ IDEA ou Eclipse, Java 17+ installé, Maven ou Gradle configuré)
- Connaissances de base en mathématiques (statistiques, algèbre linéaire)
- Notions de Machine Learning bienvenues mais pas obligatoires

Qu'est-ce que l'IA ?

- L'IA est un ensemble de techniques visant à simuler l'intelligence humaine : apprentissage, raisonnement, perception.
- En IA classique, les modèles apprennent à partir de données pour faire des prédictions ou prendre des décisions.

Pourquoi utiliser l'IA ?

- Automatiser des tâches répétitives ou complexes
- Améliorer la précision et la réactivité dans la chaîne logistique
- Détecter des anomalies et anticiper des problèmes

IA Classic vs IA Générative

IA classique	IA générative
Basée sur des données structurées	Génère du texte, du code, des images
Prédit, classifie, recommande	Crée de nouvelles données similaires
Ex : régression, classification	Ex : ChatGPT, DALL·E, Copilot

En résumé :

- **IA Classique** : Réponds à partir d'un ensemble connu de données → prédiction, classification, décision.
- **IA Générative** : Crée quelque chose de nouveau, plausible, **mais pas forcément vrai**, à partir des patterns qu'elle a appris.

Modèles :

- **Machine Learning (ML)** : Branche de l'IA fondée sur les algorithmes d'apprentissages à partir de données
- **Deep Learning (DL)** : Sous-branche du ML basée sur les réseaux de neurones profonds

Types d'apprentissage

- **Supervisé** : Modèles entraînés à partir de données étiquetées (classification, régression)
- **Non-supervisé** : Recherche de structures (clustering, réduction de dimension)
- **Apprentissage par renforcement** : Apprentissage par essais/erreurs

❓ Algorithme d'apprentissage

❓ Qu'est-ce qu'un algorithme d'apprentissage

Un algorithme d'apprentissage est une méthode mathématique utilisée pour créer un modèle prédictif à partir de données. Il permet à la machine d'apprendre automatiquement à résoudre un problème sans qu'on lui donne les règles à la main.

❓ Comment ça fonctionne ?

- ❖ On fournit des exemples => heure = 8h, distance = 120 km, pluie = oui → retard = oui
- ❖ Analyse les données pour trouver des motifs (relations entre les entrées (features) et la sortie (label))
- ❖ Ajuste les paramètres pour réduire l'écart entre prédictions et réalité.
- ❖ Généralisation => il peut prédire des résultats sur de nouveaux cas

❓ Un exemple simple

On veut prédire le retard d'un colis. La régression logistique apprend une formule ($\text{proba_retard} = f(\text{heure}, \text{distance}, \text{pluie})$) et l'ajuste automatiquement en comparant prédictions et réalité.

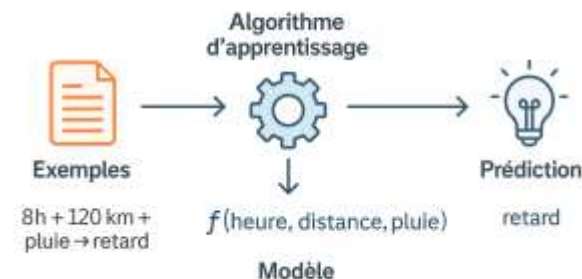
❓ Pourquoi c'est utile ?

Comme on ne peut pas écrire toutes les règles du réel, l'algorithme les déduit en observant les données :

- De s'adapter à des environnements complexes
- D'automatiser des tâches autrefois réservées aux humains
- D'apprendre en continu, au fur et à mesure qu'on collecte des données

Qu'est-ce qu'un algorithme d'apprentissage?

Apprentissage automatique à partir de données



🔧 Implémentation d'une régression linéaire supervisée

//👤 Objectif pédagogique => Implémenter une régression linéaire supervisée

```
public class LinearRegressionSupervised {
    public static void main(String[] args) { //On apprend à prédire une sortie Y à partir d'une entrée X avec une fct du type  $Y = a * X + b$ .
        double[] xData = {6, 8, 10, 12}; // Données d'entraînement => X: heure
        double[] yData = {0, 0, 1, 1}; // Données d'entraînement => Y: retard oui=1 / non=0
        double a = 0.0; // pente
        double b = 0.0; // intercept
        double learningRate = 0.01;

        for (int epoch = 0; epoch < 1000; epoch++) { // Entraînement (gradient descent)
            double totalErrorA = 0;
            double totalErrorB = 0;

            for (int i = 0; i < xData.length; i++) {
                double x = xData[i];
                double y = yData[i];
                double prediction = a * x + b;
                double error = prediction - y;
                totalErrorA += error * x;
                totalErrorB += error;
            }

            a -= learningRate * totalErrorA / xData.length; // Mise à jour de la pente
            b -= learningRate * totalErrorB / xData.length; // Mise à jour de la constante intercept
        }
        System.out.printf("📊 Modèle entraîné :  $y = %.3f * x + %.3f$ ", a, b); // Affichage du modèle appris

        double testHour = 11; // Test : prédire le retard pour 11h
        System.out.printf("🕒 Probabilité de retard à %.0fh : %.2f", testHour, a * testHour + b);
    }
}
```

Pipeline typique de Machine Learning :

- Collecte et exploration des données
- Prétraitement des données (nettoyage, normalisation)
- Encodage des variables
- Division train/test
- Entraînement du modèle
- Évaluation (précision, matrice de confusion, f1-score)
- Export / déploiement
- Prédiction sur jeux de données

IA dans la logistique

- Prévvision de retards de livraison
- Optimisation des tournées
- Maintenance prédictive
- Analyse de comportements (chauffeurs, flotte, trafic)

Avantages clés

- Réduction des coûts
- Amélioration du service client
- Détection proactive des risques

Objectif du modèle

Prédire si une livraison sera en retard à partir des caractéristiques d'une course.

Données d'entrée simulées (CSV)

- heure_depart (ex: 08:30)
- distance_km
- pluie (oui/non)
- jour_semaine (lundi, mardi...)
- vehicule_type (camion, fourgon...)
- retard (oui/non)

Objectif du modèle

Prédire si une livraison sera en retard à partir des caractéristiques d'une course.

Déroulement du TP

✓ Chargement des données

- Lire le fichier CSV fourni (livraison_retards_dataset.csv)
- Afficher les premières lignes pour validation

✓ Prétraitement

- Encoder les variables catégorielles (*jour, type de véhicule, pluie, heure de départ, distance parcouru*)
- Convertir `heure_depart` en format numérique

✓ Division Train/Test

- Utiliser 80% des données pour l'entraînement, 20% pour les tests

✓ Entraînement du modèle

- Utiliser un classifieur simple de Tribuo : `LogisticRegressionTrainer`, `CARTTrainer...`
- Générer le modèle

✓ Évaluation

- Calculer l'accuracy, matrice de confusion, f1-score
- Interpréter les erreurs possibles

Fichier CSV (livraison_retards_dataset.csv)

```
id_course,heure_depart,distance_km,pluie,jour_semaine,vehicule_type,retard
1000,16:00,11,non,mardi,camion,non
1001,08:00,178,oui,vendredi,camionnette,oui
1002,06:00,28,oui,mardi,fourgon,non
1003,15:00,148,oui,vendredi,camionnette,oui
1004,09:45,155,non,lundi,camion,oui
1005,17:45,92,non,mardi,camion,oui
1006,18:30,31,oui,jeudi,camion,oui
1007,11:30,159,non,lundi,fourgon,oui
1008,13:00,101,oui,vendredi,camionnette,oui
1009,16:30,152,oui,lundi,camion,oui
1010,16:15,79,oui,mardi,camion,oui
1011,12:30,121,non,mardi,camionnette,oui
1012,11:15,176,non,lundi,fourgon,oui
1013,16:15,141,oui,mardi,camionnette,oui
1014,12:30,168,oui,mercredi,camion,oui
1015,09:00,85,non,mercredi,camion,non
1016,09:30,59,non,jeudi,fourgon,non
1017,13:15,72,oui,mardi,fourgon,oui
1018,14:30,196,non,vendredi,camionnette,oui
1019,11:15,40,non,lundi,camion,non
1020,07:15,165,oui,jeudi,fourgon,oui
1021,07:45,102,non,vendredi,camionnette,oui
1022,14:00,179,oui,vendredi,camionnette,oui
1023,18:30,33,non,jeudi,camion,non
1024,13:00,189,non,vendredi,camion,oui
1025,14:00,165,non,vendredi,fourgon,oui
1026,09:15,100,oui,vendredi,fourgon,oui
1027,06:30,130,oui,lundi,camionnette,oui
1028,18:30,66,oui,mardi,fourgon,oui
1029,07:00,192,non,lundi,fourgon,oui
1030,18:15,37,non,vendredi,camion,non
1031,10:45,59,oui,mercredi,camionnette,oui
```

Préparer un projet Maven :

- Ajoutez les dépendances suivantes dans le fichier pom.xml :

```
<tribuo.version>4.3.2</tribuo.version>
<mockito.version>5.14.2</mockito.version>

<!-- Exercise : Introduction IA -->
<dependency>
  <groupId>org.tribuo</groupId>
  <artifactId>tribuo-all</artifactId>
  <version>${tribuo.version}</version>
  <type>pom</type>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <version>${mockito.version}</version>
</dependency>
```

```

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class LogistiqueLMIATests {
    private static final String fileName = "livraison_retards_dataset.csv";
    private static final String newFileName = "livraison_retards_dataset_converted.csv";
    private static final String modelFile = "livraison_regressor.ser";
    private static final Path input = Paths.get("src", "main", "resources", fileName);
    private static final Path output = Paths.get("src", "main", "resources", newFileName);
    private static final Path MODEL_PATH = Paths.get("src", "main", "resources", modelFile);
    private static LabelFactory labelFactory;
    private static LinkedHashMap<String, FieldProcessor> fieldProcessors;

    @BeforeAll
    public static void setUp() {
        labelFactory = new LabelFactory(); // Définir le label factory
        fieldProcessors = new LinkedHashMap<>(); // Définir les extracteurs de colonnes
        configFile(); // encodage des données
    }

    @AfterAll
    public static void tearDown() { // Nettoyage des ressources
        if (output.toFile().exists()) {
            boolean deleted = output.toFile().delete();
            assertTrue(deleted, "Le fichier converti doit être supprimé après les tests");
        }
        if (MODEL_PATH.toFile().exists()) {
            boolean deleted = MODEL_PATH.toFile().delete();
            assertTrue(deleted, "Le fichier du modèle doit être supprimé après les tests");
        }
    }

    ..... Vos Méthodes .....
}

```

Encoder les variables catégorielles (jour, type de véhicule, pluie, heure_decimal, distance_km)

```
private static void configFile() {  
  
    // nouveau champ calculé prétraité  
    fieldProcessors.put("heure_decimal", new DoubleFieldProcessor("heure_decimal"));  
  
    // distance_km est déjà numérique  
    fieldProcessors.put("distance_km", new DoubleFieldProcessor("distance_km"));  
  
    // pluie, jour_semaine, vehicule_type = colonnes catégorielles  
    fieldProcessors.put("pluie", new IdentityProcessor("pluie"));  
    fieldProcessors.put("jour_semaine", new IdentityProcessor("jour_semaine"));  
    fieldProcessors.put("vehicule_type", new IdentityProcessor("vehicule_type"));  
  
    // Le processeur de la colonne de sortie (retard)  
    FieldResponseProcessor<Label> responseProcessor =  
        new FieldResponseProcessor<>("retard", "non", LabelFactory);  
  
    // Création du RowProcessor avec les éléments définis  
    rowProcessor = new RowProcessor<>(responseProcessor, fieldProcessors);  
}
```

```
@Test
@Order(1)
void prepareDatasets() {
    // Convertir heure_depart en format numérique
    HeureDepartPreprocessor.convertPreprocessor(input, output);
    assertTrue(output.toFile().exists(), "Le fichier converti doit exister");
    assertTrue(output.toFile().length() > 0, "Le fichier converti ne doit pas être vide");
}

private static RowProcessor<Label> rowProcessor;
private static CSVDataSource<Label> dataSource;

@Test
@Order(2)
void loadDatasets() throws IOException {
    // Charger le fichier CSV fourni (livraison_retards_dataset.csv)
    dataSource = new CSVDataSource<>(
        Paths.get("src", "main", "resources", newFileName),
        rowProcessor,
        true // skip header
    );
    assertNotNull(dataSource, "La source de données ne doit pas être null");
    assertFalse(dataSource.toString().isEmpty(), "La source de données doit contenir des données");
}
```

```

@Test
@Order(3)
void splitTrainTest() { // Split train/test => Utiliser 80% des données pour l'entraînement, 20% pour les tests
    var splitter = new TrainTestSplitter<>(dataSource, 0.8, 42L);
    train = new MutableDataset<>(splitter.getTrain());
    test = new MutableDataset<>(splitter.getTest());
}

@Test
@Order(4)
void training() { // Entraînement du modèle
    var trainer = new LogisticRegressionTrainer();
    model = trainer.train(train);
}

@Test
@Order(5)
void evaluator() { // Évaluation => Calculer l'accuracy, matrice de confusion, f1-score
    var evaluator = new LabelEvaluator();
    LabelEvaluation evaluation = evaluator.evaluate(model, test);
    System.out.println("Résultats :");
    System.out.println(evaluation.toString());
}

@Test
@Order(6)
void saveModel() throws Exception { // Sauvegarde du modèle
    try (ObjectOutputStream objectOutputStream = new ObjectOutputStream(new FileOutputStream(MODEL_PATH.toFile())) {
        objectOutputStream.writeObject(model);
    }
}

```

```

@Test
@Order(7)
void predictor() throws Exception {
    File modelFile = MODEL_PATH.toFile();
    Model<Label> loadedModel = null;
    ObjectInputStream objectInputStream = new ObjectInputStream(new FileInputStream(modelFile));
    loadedModel = (Model<Label>) objectInputStream.readObject();

    // System.out.println("Q Features attendues par le modèle :");
    // loadedModel.getFeatureIDMap().forEach(name -> System.out.println(" - " + name));

    Example<Label> example = new ArrayExample<>(new Label("non"));
    // l'exemple doit avoir les mêmes nom de features que ceux générés par le modèle (FieldProcessor)
    example.add(new Feature("distance_km@value", 120.0));
    example.add(new Feature("heure_decimal@value", 8.0));
    example.add(new Feature("pluie@non", 0.0));
    example.add(new Feature("jour_semaine@mercredi", 2.0));
    example.add(new Feature("vehicule_type@camionnette", 1.0));

    // === Prédiction sur l'exemple ===
    prediction = loadedModel.predict(example);
    System.out.println("👁 Prédiction : " + prediction.getOutput());
}

```

Objectif :

- à partir des données fournis créer un nouveau modèle permettant de réaliser des prédictions sur la pluie en fonction des jours de la semaine et des retards.

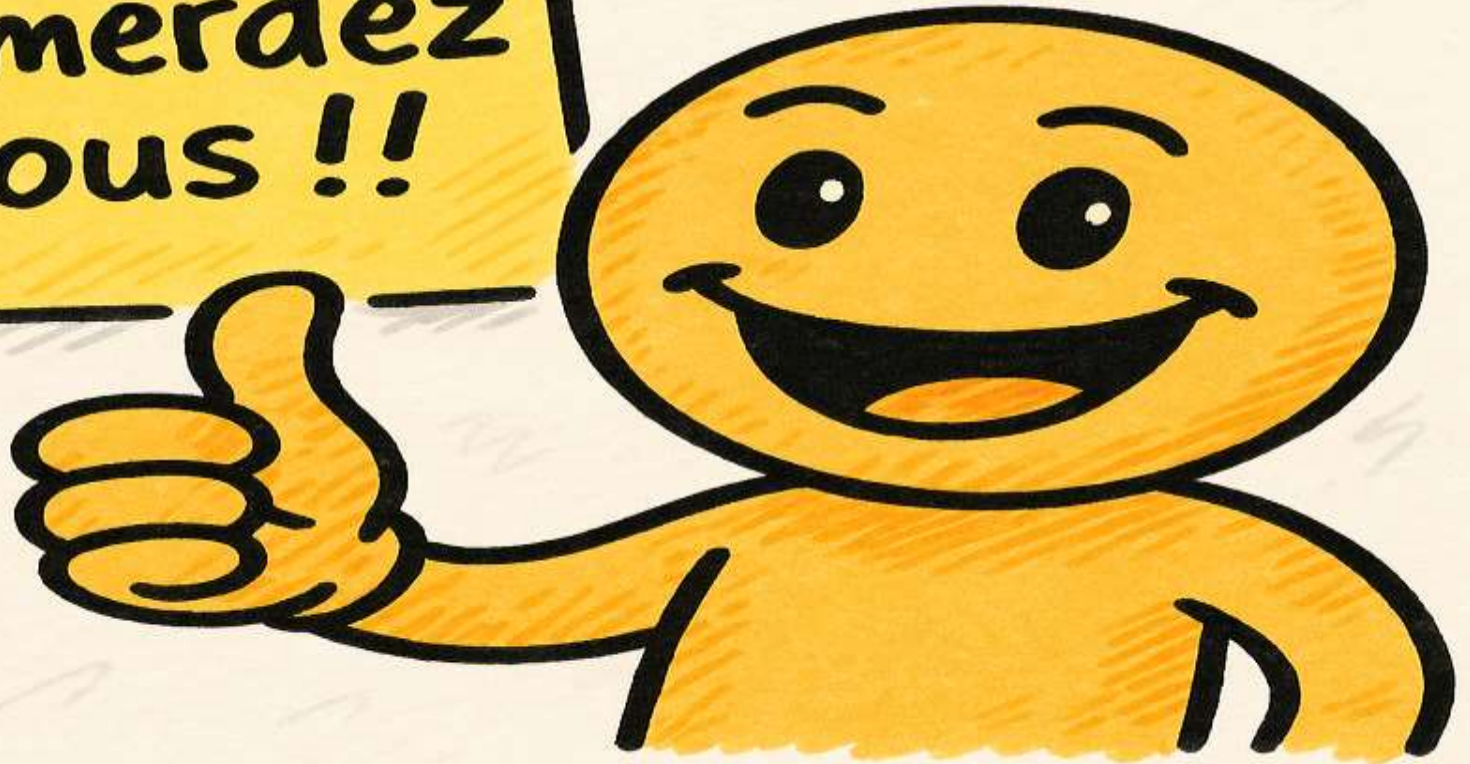
Implémentation :

- Modifier les valeurs catégorielles
- Générer un nouveau modèle de prédiction
- Calculez et Interprétez => accuracy, matrice de confusion, f1-score
- Calculez et Interprétez l'arbre de probabilités
- Réalisez une prédiction sur l'échantillon suivant :
 - jour_semaine => Vendredi
 - retard => Oui

💡 Solution



**Démerdez
vous !!**



🔍 Définition :

C'est le **pourcentage de prédictions correctes** parmi toutes les prédictions effectuées.

📄 Formule :

Accuracy = (nombre de prédictions correctes) / (nombre total de prédictions)

🔗 Exemple :

Sur 100 livraisons :

- 80 prédictions étaient justes
- 20 étaient fausses

👉 **Accuracy = 80%**

👉 **Avantage :** Facile à comprendre, utile quand les classes sont bien équilibrées.

🔍 Définition :

Tableau qui te montre où le modèle se trompe, en séparant :

- les retards correctement détectés
- les faux retards détectés à tort
- les retards manqués
- les livraisons normales bien classées

📊 Exemple (binaire : oui / non) :

	Prédit : retard	Prédit : non-retard
Réel : retard	40 vrais positifs (bien prédit “retard”)	10 faux négatifs (retard ignoré)
Réel : non-retard	5 faux positifs (fausse alerte)	45 vrais négatifs (bien prédit “non-retard”)

🔍 Définition :

C'est une **moyenne entre la précision et le rappel**. Il donne une idée plus juste de la performance quand les données sont **déséquilibrées** (par exemple, peu de retards dans beaucoup de livraisons).

📐 Formule :

$$F1 = 2 * (\text{précision} * \text{rappel}) / (\text{précision} + \text{rappel})$$

- **Précision** : parmi les livraisons prédites “en retard”, combien l’étaient vraiment ?
- **Rappel** : parmi les vraies livraisons en retard, combien ont été détectées ?

✓ En résumé

🎯 Exemple :

- Précision : 80%
 - Rappel : 50%
- 👉 F1-score ≈ **61%**

Indicateur	À quoi il sert ?
Accuracy	Te dire globalement combien de fois le modèle ne se trompe pas
Matrice de confusion	Te montrer où il se trompe (sur les retards ou sur les livraisons normales)
F1-score	T'indique si le modèle est équilibré entre fausses alertes et oublis

Smile (Statistical Machine Intelligence & Learning Engine)

- Algorithmes classiques (régression, arbres, SVM, clustering)
- API simple et rapide
- Compatible avec Java 8+

Tribuo (Oracle Labs)

- Framework complet ML en Java
- Gestion des pipelines, persistances, et expérimentations
- Supporte classification, régression, clustering, NLP

Autres bibliothèques

- **DL4J (DeepLearning4J)** : Deep learning en Java
- **Weka** : Interface graphique + API Java historique
- **ND4J** : Librairie de calcul numérique pour Java (backend de DL4J)
- Spring AI, Langchain4J-CDI et Quarkus-Langchain4J